

Few-shot Structural Neural Network Pruning Using Computational Graph Information

Maksim Ivanov¹[0009–0008–0025–8892] and Oleg Bakhteev²[0000–0002–6497–3667]

¹ Moscow Institute of Physics and Technology, Russia
`ivanov.mo@phystech.edu`

² Skolkovo Institute of Science and Technology, Moscow, Russia
`bakhteev.o@gmail.com`

Abstract. The paper investigates the problem of structural pruning in neural networks. Structural pruning is the procedure of removing entire groups of parameters from a neural network, for example, filters in CNNs or skip connections. An appropriate pruning strategy can improve both generalization performance and inference efficiency. The main difficulty of structural pruning is that removing a layer requires removing or consistently modifying all dependent layers, especially across arbitrary or novel architectures.

The proposed method is based on the analysis of the deep learning computation graph and the estimation of the information flow propagated through it. By preserving the local dependency structure of the loss function, the method enables the estimation of the importance of operations in the computation graph in a few-shot setting, i.e., using several forward passes of a data subset through the analyzed model. To demonstrate the effectiveness of the proposed method, we conduct multiple experiments on randomly generated neural networks using the MNIST dataset, showing that our approach outperforms existing pruning methods. Furthermore, our method achieves 60.4% accuracy after pruning 43.9% of the parameters of ResNet-18 on the CIFAR-10 classification task without fine-tuning.

Keywords: Structural pruning · Surrogate graph model · Few-shot

1 Introduction

Training neural networks requires increasingly large amounts of computational resources [21]. Developing and training models with novel architectures can be particularly challenging under limited computational budgets [22]. In addition, edge computing applications require neural network inference to be performed on resource-constrained portable devices [1]. Consequently, there is a growing need to improve both the generalization performance and the inference efficiency of neural networks. One of the approaches to addressing the issues mentioned above is model pruning.

By pruning, we mean the task of reducing the size of a network by removing parameters [14]. Pruning is generally classified into unstructured [16, 24] and

structured [26, 18] approaches. In the first case, individual irrelevant weights are set to zero, whereas in the second case, entire groups of parameters are removed from the neural network. Several methods for unstructured pruning have been proposed [13, 8, 28, 7]. Most of them rely on removing weights according to an importance criterion, for example, weight magnitude [7, 17], norm-based criteria [11], or estimated loss change [20, 8].

Although unstructured pruning helps reduce the computational cost of inference [7], pruning models with novel architectures remains challenging, especially when dependent layers must be removed simultaneously. This issue is investigated in [5], where structural pruning is performed by constructing a dependency graph to explicitly model inter-layer dependencies and to group coupled parameters for joint pruning. Other works review the structural pruning problem and propose methods that either do not use dataset information [23, 27] or rely only on a small number of samples [23, 9]. The goal is to perform pruning in a zero-shot setting, i.e., without fine-tuning [2, 15]. However, some existing methods are designed for specific architectures, which poses a significant obstacle to applying such algorithms across diverse problems [6, 25].

In this paper, we investigate the problem of structural pruning. We represent the computation graph as a directed graph whose vertices correspond to network layers (see Fig. 1). The proposed method is based on analysis the deep learning computation graph and estimating the information flow propagated through it. Our goal is to perform structural pruning for arbitrary neural network architectures without restricting the method to a specific family of models. The method enables the estimation of the importance of operations in the computation graph in a few-shot setting, i.e., using only a small subset of data passed through the analyzed model.

The computational experiments are conducted on the MNIST dataset [3]. The proposed method outperforms surrogate-based baselines that do not replicate the structure of the original model (fully connected network, linear regression and magnitude pruning [7] and the method using the first and second-order Taylor expansions [19]), achieving $1.5\times$ higher correlation metrics.

This improvement is further illustrated using correlation coefficients and scatter plots.³

Our main contributions are:

1. We propose a surrogate-based structural pruning method that explicitly incorporates information from the neural network computation graph. The method represents the model as a directed computational graph and introduces structural constraints on the surrogate model derived from the information flow within this graph.
2. We theoretically demonstrate that the proposed surrogate graph model accurately approximates the true error function.
3. We empirically demonstrate that the proposed graph-based constraints improve surrogate reliability in the few-shot setting, enabling more stable and interpretable estimation of operation importance for structural pruning.

³ The code will be published in the camera-ready version.

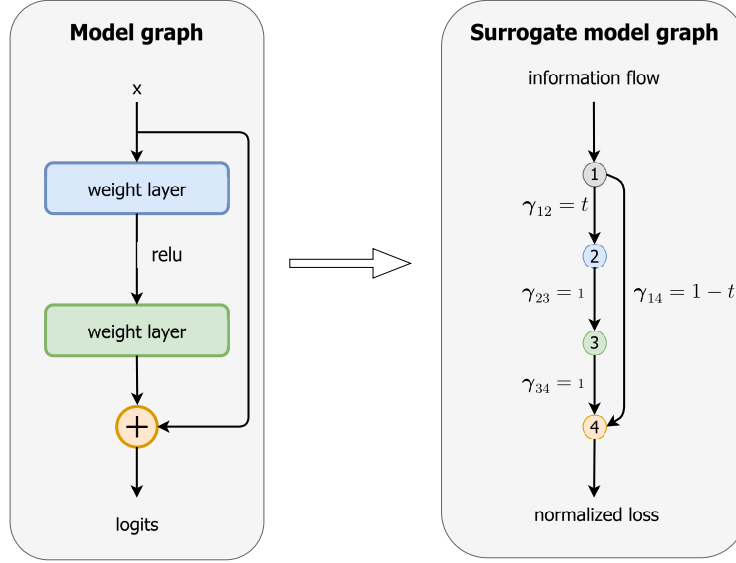


Fig. 1. A schematic illustration of a residual connection and the corresponding surrogate model graph. A coefficient γ_{ij} is assigned to each edge; the collection of these coefficients determines the optimized graph structure.

4. We validate the proposed approach on multiple models trained on the MNIST dataset [3]. To assess the generality of the method, we consider randomly generated computational graphs inspired by neural architecture search [29] search spaces and show that incorporating computation graph information leads to improved performance compared to surrogate models that ignore the original model structure. We also implement an importance interface for the DepGraph method [5]. Furthermore, the proposed method achieves 60.4% accuracy after pruning 43.9% of the parameters of ResNet-18 on the CIFAR-10 classification task without fine-tuning.

2 Problem statement

We address the problem of structural pruning in deep neural networks formulated in terms of their computation graphs. We define the **computation graph** of a neural network as a directed graph $G_{\text{computation}} = (V, E)$, where each vertex $v_i \in V$ corresponds to a layer (or computational module) of the network, and each directed edge $e_{ij} \in E$ represents the data flow from the output of layer v_i to the input of layer v_j . Formally,

$$e_{ij} \in E \Leftrightarrow \mathbf{h}_j = \mathbf{f}_j(\mathbf{h}_i, \mathbf{w}_j), \quad (1)$$

where $\mathbf{h}_i$ denotes the activation produced by vertex v_i , and $\mathbf{f}_j$ is the transformation implemented at vertex v_j . In this formulation, removing a vertex corre-

sponds to eliminating the entire layer from the network, whereas removing an edge corresponds to removing a data dependency between two layers (e.g., in residual or multi-branch architectures).

We consider the scenario of **edge removal** — deleting selected connections $e_{ij} \in E$ in order to reduce memory consumption and computational cost, possibly at the expense of predictive accuracy. Alternatively, one may prune either vertices, which correspond to layers in the source model, respectively. These pruning strategies can be reduced to one another.

Let the dataset be denoted by $\mathfrak{D} = (\mathbf{X}, \mathbf{y})$, where $\mathbf{X} \in \mathbb{R}^{m \times n}$ is the matrix of input features and $\mathbf{y} \in \mathbb{R}^m$ is the vector of target variables. Let $\mathbf{w} \in \mathbb{R}^k$ be the vector of model parameters. The model is trained by minimizing the error function

$$S = S(\mathbf{w} \mid \mathfrak{D}, f). \quad (2)$$

After pruning, we obtain a modified parameter vector $\mathbf{w}' \in \mathbb{R}^k$, $\mathbf{w}' \neq \mathbf{w}$, in which certain groups of parameters are set to zero, corresponding to removed edges or vertices in the computation graph. The challenge of structural pruning lies in estimating the importance of parameter groups in terms of their contribution to the network information flow and the final predictive performance.

The following baselines can be found in the literature:

1. **Loss-based pruning** — directly measuring the post-pruning error difference and selecting $\mathbf{w}'$ to minimize the performance drop:

$$\min_{\mathbf{w}' \in \mathbb{R}^k} |S(\mathbf{w}' \mid \mathfrak{D}, f) - S(\mathbf{w} \mid \mathfrak{D}, f)|. \quad (3)$$

This objective leads to a combinatorial optimization problem and is therefore computationally intractable in practice [19].

2. **Magnitude-based pruning** — removing parameters with the smallest magnitudes. This approach is based on the assumption that the magnitude of a parameter correlates with its importance [7]. In practice, the L_1 -norm is often used to estimate parameter importance.
3. **Taylor-based pruning** — estimating the impact of pruning using a second-order Taylor expansion of the error function around $\mathbf{w}$:

$$S(\mathbf{w}') \approx S(\mathbf{w}) + \langle \nabla S(\mathbf{w}), \Delta \mathbf{w} \rangle + \frac{1}{2} \Delta \mathbf{w}^\top \mathbf{H} \Delta \mathbf{w}, \quad (4)$$

where $\nabla S(\mathbf{w})$ is the gradient of the error function and $\mathbf{H} = \nabla^2 S(\mathbf{w})$ is the corresponding Hessian matrix [19].

The use of local pruning methods often leads to a degradation in performance when multiple layers are pruned simultaneously [26]. Therefore, **Magnitude-based pruning** and **Taylor-based pruning** are not well suited for large-scale structural pruning.

The problem is therefore to design an importance estimation strategy for parameter groups that enables effective structural pruning, i.e., maximizes the reduction in model size and computational complexity while keeping the increase in the error function within acceptable bounds.

3 Structural pruning using computation graph information

Let the dataset be denoted by

$$\mathfrak{D} = \mathfrak{D}_{\text{train}} \cup \mathfrak{D}_{\text{val}}. \quad (5)$$

We employ the loss-based pruning method defined by the optimization problem (3).

The proposed pruning algorithm constructs a **surrogate graph model** $G(\gamma, \mathbf{M})$ that replicates the structure of the original neural network. The graph is derived from the computation graph of the model. For each edge $e_{ij} \in E$, introduce a continuous importance coefficient $\gamma_{ij} \in [0, 1]$. Additionally, define a binary edge mask $\mathbf{M} \in \mathbb{Z}_2^{|E|}$ indicating whether an edge is pruned (see Fig. 2). Formally,

$$\mathbf{M}_{ij} = 0 \Leftrightarrow \gamma_{ij} = 0. \quad (6)$$

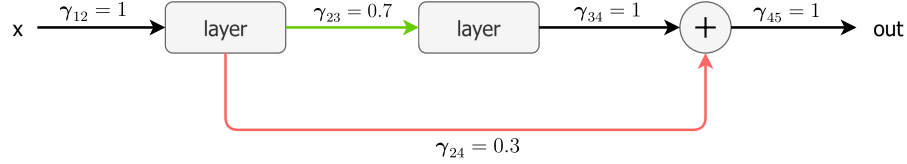


Fig. 2. Simplified schematic of a residual connection. A coefficient γ_{ij} is assigned to each edge; smaller values of γ_{ij} indicate lower edge importance.

Lemma 1. *The graph model $G(\gamma, \mathbf{M})$ can be represented as*

$$G(\gamma, \mathbf{M})[\mathbf{x}] = \mathbf{x} \cdot \sum_{p \in P} \prod_{(i,j) \in p} (\gamma_{ij} \cdot \mathbf{M}_{ij}), \quad (7)$$

where P — the set of all paths in the graph.

Proof. Consider an arbitrary path $p \in P$. During the forward pass, the input $\mathbf{x}$ is multiplied by the edge weights γ_{ij} and the corresponding masks $\mathbf{M}_{ij}$ along the path p :

$$\mathbf{x} \cdot \prod_{(i,j) \in p} (\gamma_{ij} \cdot \mathbf{M}_{ij}). \quad (8)$$

Summing these contributions over all paths yields the desired expression.

We prove the following theorem describing the local behavior of the model.

Theorem 1. *Let the loss function S be normalized such that $S \in [0, 1]$. For any fixed vertex $i \in V$ let $E_i = \{(i, j) \in E\}$ denote the set of outgoing edges from vertex i , and let $\mathbf{M}_i = \{\mathbf{M}_{ij} \mid (i, j) \in E_i\}$ be the corresponding subset of mask variables, with all other masks in the graph fixed to 1. Then, the error function S can be locally approximated by the graph surrogate model:*

$$S(\mathbf{w}, \mathbf{M}_i \mid \mathfrak{D}, f) = C + \sum_{(i,j) \in E_i} \gamma_{ij} \mathbf{M}_{ij} + o(\|\mathbf{M}_i - \mathbf{1}\|), \quad (9)$$

where C is a constant, and

$$\gamma_{ij} = \frac{\partial S}{\partial \mathbf{M}_{ij}}. \quad (10)$$

Proof. According to Lemma 1, the graph model $G(\gamma, \mathbf{M})$ is structured as a sum over paths. Since the network is a directed acyclic graph, any path $p \in P$ can contain at most one outgoing edge from the vertex i . Consequently, when all masks outside of E_i are fixed to 1, the product over each path contains at most one variable from $\mathbf{M}_i$, meaning the graph model locally reduces to a linear form:

$$G(\gamma, \mathbf{M}_i) = C' + \sum_{(i,j) \in E_i} \gamma_{ij} \mathbf{M}_{ij}, \quad (11)$$

where C' is a constant.

Applying a first-order Taylor expansion to the error function S around the point $\mathbf{M}_i = \mathbf{1}$ yields:

$$S(\mathbf{w}, \mathbf{M}_i \mid \mathfrak{D}, f) = S(\mathbf{w}, \mathbf{1} \mid \mathfrak{D}, f) + \sum_{(i,j) \in E_i} \frac{\partial S}{\partial \mathbf{M}_{ij}} (\mathbf{M}_{ij} - 1) + o(\|\mathbf{M}_i - \mathbf{1}\|). \quad (12)$$

By expanding the terms and grouping the components independent of $\mathbf{M}_i$ into a constant C , we can match the linear expansion with the graph surrogate model parameters:

$$\begin{aligned} S(\mathbf{w}, \mathbf{M}_i \mid \mathfrak{D}, f) &= \left(S(\mathbf{w}, \mathbf{1} \mid \mathfrak{D}, f) - \sum_{(i,j) \in E_i} \frac{\partial S}{\partial \mathbf{M}_{ij}} \right) \\ &+ \sum_{(i,j) \in E_i} \frac{\partial S}{\partial \mathbf{M}_{ij}} \mathbf{M}_{ij} + o(\|\mathbf{M}_i - \mathbf{1}\|). \end{aligned} \quad (13)$$

In conclusion, define $C = S(\mathbf{w}, \mathbf{1} \mid \mathfrak{D}, f) - \sum_{(i,j) \in E_i} \frac{\partial S}{\partial \mathbf{M}_{ij}}$ and set $\gamma_{ij} = \frac{\partial S}{\partial \mathbf{M}_{ij}}$.

Proposition 1. *We impose the following normalization constraint on the outgoing edges of each vertex:*

$$\sum_{j: (i,j) \in E} \gamma_{ij} = 1. \quad (14)$$

The method is based on optimizing this surrogate graph model. The final optimization problem is formulated as

$$\begin{aligned} \gamma = \arg \min_{\gamma \in [0,1]^{|E|}} \quad & \mathbb{E}_{\mathbf{M}} \|G(\gamma, \mathbf{M}) - S(\mathbf{w} \mid \mathfrak{D}, f)\|_2^2 \\ \text{s.t.} \quad & \sum_{j: e_{ij} \in E} \gamma_{ij} = 1, \end{aligned} \quad (15)$$

where $\mathbf{M}$ is sampled uniformly from the set $\mathcal{M}$ consisting of: the all-zero mask, the all-one mask, and all possible masks with exactly one zero element (iterating over all edges $e_{ij} \in E$).

4 Computational experiment

4.1 Settings

This section focuses on the choice of neural network architecture. To conduct the experiments, a handwritten digit classification task based on the MNIST dataset [3] was considered.

To solve this task, we implemented an algorithm for generating a graph neural network architecture. The procedure follows an approach similar to neural architecture search [29]. Let `MLPblock` denote a node consisting of a linear layer followed by the ReLU activation function. Formally,

$$\text{MLPblock}(\mathbf{x}) = \text{ReLU}(\text{Linear}(\mathbf{x})). \quad (16)$$

Define `node_list` as the list of constructed nodes. Initially, `node_list` contains two nodes: the input node and a node that maps the input to a latent representation. Then, `n_iterations` iterations of the algorithm are performed. Each iteration consists of four steps.

- Step 1.** Independently select two parent nodes from `node_list`.
- Step 2.** Apply a `MLPblock` to the first parent node selected in Step 1 and append the resulting node to `node_list`.
- Step 3.** Apply a `MLPblock` to the second parent node selected in Step 1 and append the resulting node to `node_list`.
- Step 4.** Apply a `MLPblock` to combine the outputs obtained in Steps 2 and 3, thereby creating a new node, and append it to `node_list`.

After completing all iterations, the algorithm aggregates all terminal (hanging) nodes and applies a final classifier for the MNIST task.

Depending on the value of `n_iterations`, the resulting architecture contains a different number of edges (see Figs. 3, 4).

- **L1**: Magnitude pruning [7].
- **Taylor expansion**: The method using the first and second-order Taylor expansions [19].

The **surrogate graph model** is trained for 3000 epochs, while the **FCN model** is trained for 1000 epochs. Both models are optimized using the mean squared error (MSE) loss and the Adam optimizer with learning rate 10^{-3} .

In the experiments, the Spearman and Kendall rank correlation coefficients between the true and predicted error values were used as evaluation metrics. These metrics measure the agreement between the corresponding rankings. Additionally, we computed the overlap between the top- $k\%$ of models ranked according to the true loss and the top- $k\%$ of models ranked according to the surrogate predictions. For a more robust evaluation, each experiment was repeated ten times, and the reported results were averaged.

4.2 Experiment results

As shown in Tables 1 and 2, the proposed method demonstrates strong performance in the few-shot setting. It significantly outperforms the baseline models and provides informative estimates of edge importance.

Table 1. Average Spearman correlation coefficient over 10 runs.

Model	10–15 edges	20–40 edges	50–100 edges	more than 100 edges
Graph	0.299 ± 0.235	0.296 ± 0.178	0.349 ± 0.128	0.411 ± 0.057
FCN	-0.009 ± 0.213	-0.060 ± 0.187	-0.028 ± 0.090	-0.053 ± 0.099
Linear	0.060 ± 0.187	0.201 ± 0.122	0.248 ± 0.090	0.294 ± 0.047
L1	0.547 ± 0.063	0.505 ± 0.063	0.369 ± 0.060	0.280 ± 0.081
Taylor expansion	0.119 ± 0.247	-0.052 ± 0.082	-0.136 ± 0.076	-0.204 ± 0.044

Table 2. Average Kendall correlation coefficient over 10 runs.

Model	10–15 edges	20–40 edges	50–100 edges	more than 100 edges
Graph	0.253 ± 0.202	0.242 ± 0.146	0.282 ± 0.105	0.329 ± 0.045
FCN	-0.011 ± 0.151	-0.042 ± 0.128	-0.020 ± 0.061	-0.036 ± 0.066
Linear	0.039 ± 0.135	0.137 ± 0.083	0.167 ± 0.061	0.199 ± 0.032
L1	0.401 ± 0.057	0.353 ± 0.049	0.259 ± 0.041	0.192 ± 0.057
Taylor expansion	0.094 ± 0.182	-0.033 ± 0.055	-0.090 ± 0.051	-0.136 ± 0.030

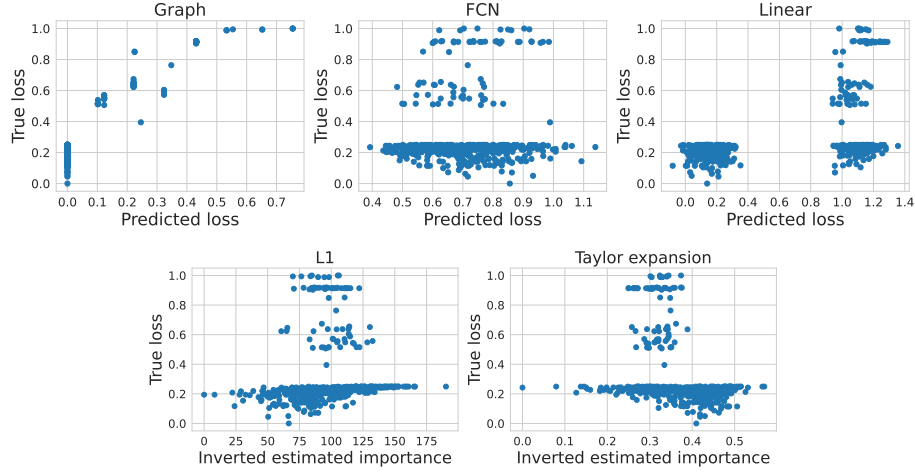


Fig. 5. Scatter plot of true and predicted error values for the baseline comparison. The parameter `n_iterations` is set to 2.

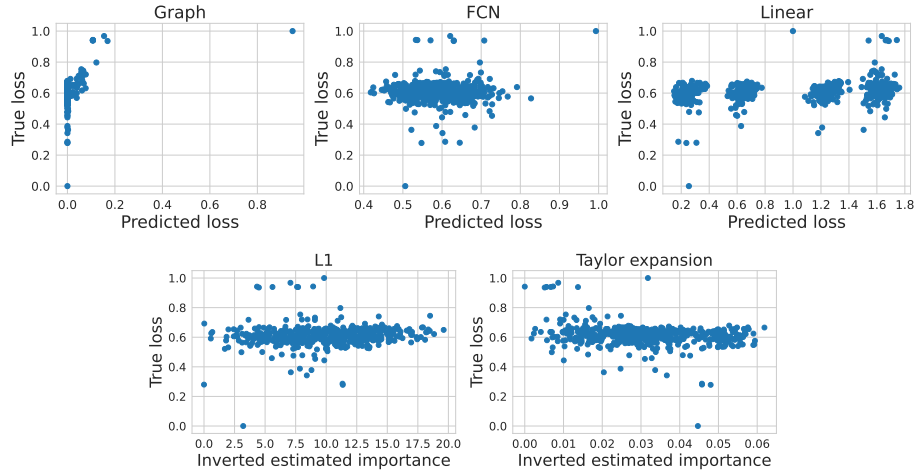


Fig. 6. Scatter plot of true and predicted error values for the baseline comparison. The parameter `n_iterations` is set to 30.

Additionally, scatter plots (Figs. 5, 6) illustrate the relationship between true and predicted error values. In all considered cases, the surrogate graph model exhibits a stronger dependence between true and predicted errors than

the baseline models. These visualizations are consistent with the correlation metrics reported above.

We compute the overlap between the top- k % of models ranked according to the true loss and the top- k % of models ranked according to the surrogate prediction. The resulting overlap ratio is used as a measure of prediction accuracy. To improve statistical stability, the experiment is repeated 10 times and the results are averaged. The results are presented in Tables 3 and 4.

Table 3. The overlap between the top-5% of models ranked according to the true loss and the models ranked according to the surrogate prediction.

Model	10–15 edges	20–40 edges	50–100 edges	more than 100 edges
Graph	62.0%	64.5%	69.3%	58.3%
FCN	2.0%	15.2%	5.2%	6.8%
Linear	18.8%	9.6%	27.2%	15.2%
L1	0.0%	1.6%	6.8%	6.8%
Taylor expansion	8.0%	10.8%	29.6%	24.4%

Table 4. The overlap between the top-10% of models ranked according to the true loss and the models ranked according to the surrogate prediction.

Model	10–15 edges	20–40 edges	50–100 edges	more than 100 edges
Graph	55.3%	54.8%	67.4%	66.9%
FCN	7.4%	16.6%	12.4%	11.4%
Linear	22.6%	15.0%	28.6%	25.8%
L1	2.0%	4.0%	12.0%	10.4%
Taylor expansion	12.6%	16.8%	30.0%	29.6%

4.3 Convolutional blocks

Let **ConvBlock** denote a node consisting of a convolutional layer (kernel = 3, padding = 1) followed by the ReLU activation function. Formally,

$$\text{ConvBlock}(\mathbf{x}) = \mathbf{ReLU}(\mathbf{Conv}(\mathbf{x})). \quad (17)$$

All other experimental settings were the same as in the previous experiment. We report the results for the 10–15 edges setup. To demonstrate the superiority of our method, we use the overlap between the top- k % of models ranked according to the true loss and the top- k % of models ranked according to the surrogate predictions. To improve statistical stability, each experiment was repeated ten times, and the reported results were averaged. The results are presented in Table 5.

Table 5. The overlap between the top-5% and top-10% of models with 10–15 edges ranked according to the true loss and the models ranked according to the surrogate prediction.

Model	top-5%	top-10%
Graph	42.6%	26.2%
FCN	4.0%	10.0%
Linear	8.4%	10.6%
L1	0.0%	0.6%
Taylor expansion	5.2%	9.2%

4.4 Ablation study

We provide an ablation study to evaluate the effectiveness of our few-shot approach for generating the training dataset for the surrogate model. In the problem formulation section, we propose sampling $\mathbf{M}$ uniformly from the set $\mathcal{M}$, which consists of the all-zero mask, the all-one mask, and all possible masks with exactly one zero element (obtained by iterating over all edges $e_{ij} \in E$). As an alternative, we independently sample each mask element from a Bernoulli distribution with probability $\frac{1}{2}$. The results in Tables 6 and 7 demonstrate a degradation in performance when this alternative mask sampling strategy is used.

Table 6. Average Spearman correlation coefficient over 10 runs.

Model	10–15 edges	20–40 edges	50–100 edges	more than 100 edges
Graph	0.299 $\pm$ 0.235	0.296 $\pm$ 0.178	0.349 $\pm$ 0.128	0.411 $\pm$ 0.057
Graph (Bern)	0.198 $\pm$ 0.212	0.336 $\pm$ 0.123	0.305 $\pm$ 0.173	0.396 $\pm$ 0.126

Table 7. The overlap between the top-5% of models ranked according to the true loss and the models ranked according to the surrogate prediction.

Model	10–15 edges	20–40 edges	50–100 edges	more than 100 edges
Graph	62.0%	64.5%	69.3%	58.3%
Graph (Bern)	53.6%	61.3%	55.6%	52.0%

4.5 The DepGraph extension

DepGraph is a method that explicitly models dependencies between layers and groups coupled parameters for pruning [5]. We implemented an importance estimator for this method. In the surrogate graph model, the vertices correspond to the Dependency Groups [5]. The graph model is constructed during initialization and subsequently used to compute importance scores offline.

We conduct experiments on ResNet-18 [10] and ViT [4] using the CIFAR-10 dataset [12]. The batch size is set to 128, the number of pre-pruning training epochs is 5, the number of fine-tuning epochs is 2, the learning rate is 0.01, the fine-tuning learning rate is 0.001, and optimization is performed using SGD. We compare the proposed method with the Taylor expansion-based pruning method [19]. The results are presented in Table 8.

Table 8. Performance before and after pruning using DepGraph.

Model	Acc. before pruning	Acc. after pruning	Acc. after fine-tuning
ResNet-18 (Graph)	79.12%	60.40%	79.37%
ResNet-18 (Taylor exp.)	79.27%	13.18%	76.86%
ViT (Graph)	53.01%	52.70%	55.86%
ViT (Taylor exp.)	53.01%	49.67%	55.66%

Model	Param. before pruning	Param. after pruning	Param. pruned
ResNet-18 (Graph)	11.17M	6.27M	43.9%
ResNet-18 (Taylor exp.)	11.17M	5.25M	53.0%
ViT (Graph)	2.69M	2.43M	9.6%
ViT (Taylor exp.)	2.69M	2.00M	25.8%

5 Discussion

1. Incorporating information from the computation graph is important for structural pruning.
2. We can consider graph information as a prior (in the context of Bayesian inference) or as an inductive bias hardwired into the model (as in our case).
3. Despite the imperfect results, we observe that the method works more effectively than the considered baselines, which can be explained by the incorporation of structural information. This is particularly important in the few-shot setting, i.e., when the model is trained on only a small number of samples.
4. Empirically, a shift toward zero is observed as the number of edges increases. However, for pruning it is more important to preserve the ordering of importance scores than their absolute values. This shift can be corrected in a post-processing step.

6 Conclusion

This paper investigated the problem of structural pruning in neural networks. The proposed method is based on analyzing the deep learning computation graph and estimating the information flow propagated through it. The method enables the estimation of the importance of operations in a computation graph in a few-shot setting. Experiments have shown that the method is superior to surrogate

models that do not use computation graph information. It is also possible to fine-tune the model after pruning, which can significantly improve its performance. Thus, the important role of computation graph information in structural pruning has been demonstrated. In future work, we plan to extend and generalize the method to channel-wise pruning (for example, filters in CNNs).

References

1. Banjanović-Mehmedović, L., Husaković, A.: Edge ai: Reshaping the future of edge computing with artificial intelligence. In: BASIC TECHNOLOGIES AND MODELS FOR IMPLEMENTATION OF INDUSTRY 4.0. pp. 133–160. Basic 4.0, Academy of Sciences and Arts of Bosnia and Herzegovina (Oct 2023). <https://doi.org/10.5644/pi2023.209.07>
2. Chen, T., Ji, B., Ding, T., Fang, B., Wang, G., Zhu, Z., Liang, L., Shi, Y., Yi, S., Tu, X.: Only train once: A one-shot neural network training and pruning framework (Jul 2021). <https://doi.org/10.48550/ARXIV.2107.07467>
3. Deng, L.: The mnist database of handwritten digit images for machine learning research [best of the web]. *IEEE Signal Processing Magazine* **29**(6), 141–142 (Nov 2012). <https://doi.org/10.1109/msp.2012.2211477>
4. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Houlsby, N.: An image is worth 16x16 words: Transformers for image recognition at scale (2021), <https://arxiv.org/abs/2010.11929>
5. Fang, G., Ma, X., Song, M., Mi, M.B., Wang, X.: Depgraph: Towards any structural pruning (Jan 2023). <https://doi.org/10.48550/ARXIV.2301.12900>
6. Frantar, E., Alistarh, D.: Sparsegpt: Massive language models can be accurately pruned in one-shot (Jan 2023). <https://doi.org/10.48550/ARXIV.2301.00774>
7. Han, S., Pool, J., Tran, J., Dally, W.J.: Learning both weights and connections for efficient neural networks (Jun 2015). <https://doi.org/10.48550/ARXIV.1506.02626>
8. Hassibi, B., Stork, D.: Second order derivatives for network pruning: Optimal brain surgeon. In: Hanson, S., Cowan, J., Giles, C. (eds.) *Advances in Neural Information Processing Systems*. vol. 5. Morgan-Kaufmann (1992)
9. He, J., Ding, Y., Zhang, M., Li, D.: Towards efficient network compression via few-shot slimming. *Neural Networks* **147**, 113–125 (2022). <https://doi.org/https://doi.org/10.1016/j.neunet.2021.12.011>, <https://www.sciencedirect.com/science/article/pii/S0893608021004846>
10. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition (2015), <https://arxiv.org/abs/1512.03385>
11. He, Y., Kang, G., Dong, X., Fu, Y., Yang, Y.: Soft filter pruning for accelerating deep convolutional neural networks (Aug 2018). <https://doi.org/10.48550/ARXIV.1808.06866>
12. Krizhevsky, A.: Learning multiple layers of features from tiny images. University of Toronto (05 2012)
13. Lecun, Y., Denker, J., Solla, S.: Optimal brain damage. vol. 2, pp. 598–605 (01 1989)
14. Liang, T., Glossner, J., Wang, L., Shi, S., Zhang, X.: Pruning and quantization for deep neural network acceleration: A survey (Jan 2021). <https://doi.org/10.48550/ARXIV.2101.09671>

15. Liu, D., Liu, X.: Pruning on-the-fly: A recoverable pruning method without fine-tuning (Dec 2022). <https://doi.org/10.48550/ARXIV.2212.12651>
16. Liu, L., Zhang, S., Kuang, Z., Zhou, A., Xue, J.H., Wang, X., Chen, Y., Yang, W., Liao, Q., Zhang, W.: Group fisher pruning for practical network compression (Aug 2021). <https://doi.org/10.48550/ARXIV.2108.00708>
17. Lubana, E.S., Dick, R.P.: A gradient flow framework for analyzing network pruning (Sep 2020). <https://doi.org/10.48550/ARXIV.2009.11839>
18. Ma, X., Fang, G., Wang, X.: Llm-pruner: On the structural pruning of large language models. In: Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S. (eds.) *Advances in Neural Information Processing Systems*. vol. 36, pp. 21702–21720. Curran Associates, Inc. (2023)
19. Molchanov, P., Mallya, A., Tyree, S., Frosio, I., Kautz, J.: Importance estimation for neural network pruning (Jun 2019). <https://doi.org/10.48550/ARXIV.1906.10771>
20. Molchanov, P., Tyree, S., Karras, T., Aila, T., Kautz, J.: Pruning convolutional neural networks for resource efficient inference (Nov 2016). <https://doi.org/10.48550/ARXIV.1611.06440>
21. Sevilla, J., Heim, L., Ho, A., Besiroglu, T., Hobbhahn, M., Villalobos, P.: Compute trends across three eras of machine learning. 2022 International Joint Conference on Neural Networks (IJCNN), Padua, Italy, 2022, pp. 1-8 pp. 1–8 (Jul 2022). <https://doi.org/10.1109/ijcnn55064.2022.9891914>
22. Tan, M., Le, Q.V.: Efficientnet: Rethinking model scaling for convolutional neural networks. *International Conference on Machine Learning*, 2019 (May 2019). <https://doi.org/10.48550/ARXIV.1905.11946>
23. Tanaka, H., Kunin, D., Yamins, D.L.K., Ganguli, S.: Pruning neural networks without any data by iteratively conserving synaptic flow. *Advances in Neural Information Processing Systems* 2020 (Jun 2020). <https://doi.org/10.48550/ARXIV.2006.05467>
24. Wang, C., Zhang, G., Grosse, R.: Picking winning tickets before training by preserving gradient flow. In *Proceedings of the 8th International Conference on Learning Representations (ICLR)*, 2020 (Feb 2020). <https://doi.org/10.48550/ARXIV.2002.07376>
25. Wang, W.T., Li, H.L., Lin, W.S., Chiang, C.M., Tsai, Y.M.: Architecture-aware network pruning for vision quality applications (Aug 2019). <https://doi.org/10.48550/ARXIV.1908.02125>
26. You, Z., Yan, K., Ye, J., Ma, M., Wang, P.: Gate decorator: Global filter pruning method for accelerating deep convolutional neural networks (Sep 2019). <https://doi.org/10.48550/ARXIV.1909.08174>
27. Yvinec, E., Dapogny, A., Cord, M., Bailly, K.: Red : Looking for redundancies for data-free structured compression of deep neural networks (May 2021). <https://doi.org/10.48550/ARXIV.2105.14797>
28. Zeng, W., Urtasun, R.: MLPrune: Multi-layer pruning for automated neural network compression (2019), <https://openreview.net/forum?id=r1g5b2RcKm>
29. Zoph, B., Vasudevan, V., Shlens, J., Le, Q.V.: Learning transferable architectures for scalable image recognition (Jul 2017). <https://doi.org/10.48550/ARXIV.1707.07012>